Schema-Aware Prompt Engineering for Text-to-SQL in Analytical Databases: A Systematic Evaluation Study

Anonymous Authors

Anonymous Institution

Abstract

We present a systematic evaluation of schema-aware prompt engineering for Text-to-SQL generation targeting ClickHouse, a columnar analytical database. We investigate four dimensions---schema format, schema scope, metadata enrichment, and example selection---across three benchmarks totaling 206 queries and 10 tables: a custom analytics dataset (150 queries, 4 tables), ClickBench (43 queries, 1 table with 105 columns), and the Star Schema Benchmark (13 queries, 5-table star schema). Through ablation analysis with bootstrap confidence intervals and cross-model validation on Claude 3.5 Sonnet and Claude Sonnet 4, we find that 41.9% of improvement comes from window function and aggregation guidance (+6.7pp), 29.4% from JOIN guidance (+4.7pp), and 28.7% from dialect-specific hints (+4.6pp). The optimal configuration---Markdown format, relevant-subset scope, column descriptions, and dynamic few-shot examples---achieves 66.7% result correctness on the primary benchmark, a +37.4pp improvement over the baseline. We compare against DAIL-SQL and validate generalization across dataset structures. We release our benchmark, evaluation framework, and all experimental artifacts.

Keywords: Text-to-SQL, Large Language Models, Prompt Engineering, Schema Linking, ClickHouse, OLAP, Database Interfaces

1. INTRODUCTION

Large Language Models (LLMs) have transformed Text-to-SQL systems, enabling users to query databases using natural language. Recent approaches leveraging frontier models have achieved execution accuracy exceeding 85% on benchmarks like Spider [yu2018spider]. However, these successes primarily target transactional databases (OLTP) with relatively simple schemas and standard SQL dialects.

Analytical databases (OLAP) present distinct challenges. Systems like ClickHouse, DuckDB, and Snowflake feature columnar storage optimized for aggregation queries, specialized functions for time-series analysis, and SQL dialect variations that generic prompting strategies fail to capture. Production OLAP deployments often involve schemas with hundreds of columns, requiring careful consideration of which schema elements to include in context-limited prompts.

A critical yet underexplored aspect of LLM-based Text-to-SQL is prompt engineering for schema presentation. When a user asks ``Show me the top 10 customers by revenue last month,'' the LLM must understand which tables exist, what columns are available, their data types, relationships, and database-specific syntax requirements. The effectiveness of SQL generation depends heavily on how this schema information is communicated.

In this paper, we present a systematic ablation study of schema-aware prompt engineering for Text-to-SQL targeting ClickHouse, investigating four dimensions of prompt design across three benchmarks and two models. Our contributions include:

- A four-dimensional ablation analysis isolating the contribution of schema format, scope, metadata, and example selection to SQL generation accuracy.
- A multi-benchmark evaluation spanning 206 queries across three datasets with distinct structural characteristics (wide single-table, normalized multi-table, star schema).
- Cross-model validation comparing Claude 3.5 Sonnet and Claude Sonnet 4, demonstrating configuration robustness across model generations.
- Actionable guidelines for practitioners, with open-source release of our benchmark and evaluation framework.

2. BACKGROUND AND MOTIVATION

2.1 Text-to-SQL with Large Language Models

Text-to-SQL systems translate natural language queries into executable SQL. The advent of LLMs shifted the field toward prompt engineering strategies that leverage pre-trained knowledge of SQL syntax and database concepts. DAIL-SQL [gao2024dailsql] demonstrated that careful prompt design achieves state-of-the-art results on Spider, but these studies predominantly target SQLite and PostgreSQL with schemas averaging 5--10 tables, leaving OLAP systems underexplored.

2.2 ClickHouse and OLAP Characteristics

ClickHouse is an open-source columnar database for real-time analytical queries, powering observability platforms and analytics systems at companies including Uber, Cloudflare, and eBay. It introduces several challenges for Text-to-SQL:

- Specialized aggregate functions: argMax(), argMin(), groupArray(), quantile()
- Time-series functions: toStartOfMonth(), toStartOfWeek(), dateDiff()

- Array and nested data types: Array(String), Nested structures
- Engine-specific behaviors: MergeTree ordering, materialized views
- Large schemas: Production deployments often exceed 100 columns per table

2.3 Research Gap

Benchmarks like Spider [yu2018spider] and BIRD [li2024bird] do not capture OLAP-specific challenges. No systematic study has evaluated schema presentation strategies for analytical databases with dialect-specific syntax. Our work addresses this gap with a multi-benchmark, multi-model evaluation.

3. METHODOLOGY

We investigate four dimensions of schema-aware prompt engineering, each with multiple strategy variations. Our evaluation follows an ablation design: we establish a baseline configuration, then vary one factor at a time (OFAT) to isolate the marginal contribution of each dimension.

3.1 Schema Representation Formats

We evaluate four formats for presenting database schema information to LLMs: Format A: CREATE TABLE (SQL DDL) --- standard SQL data definition language, including column names, types, and engine specifications. Format B: Markdown Table --- human-readable tabular format with columns for name, type, and description. Format C: JSON Schema --- structured JSON representation with explicit field semantics. Format D: Natural Language --- prose descriptions of tables and columns.

3.2 Schema Scope Strategies

For databases with large schemas, including all tables and columns may exceed context limits or dilute attention. We evaluate: Full Schema, Relevant Subset (keyword-based table selection), Progressive Expansion (iterative schema revelation), and User-Guided (explicit table specification).

3.3 Metadata Enrichment

Beyond basic schema structure, additional metadata may improve generation accuracy: column descriptions, sample values, statistics, and combinations thereof.

3.4 Example Selection Methods

We compare: zero-shot (no examples), static few-shot (fixed 3 examples), dynamic few-shot (similarity-based selection), and schema-matched (examples using overlapping tables).

3.5 Pilot Study: Initial Baseline Evaluation

Prior to the full ablation analysis, we conducted a pilot study (600 API calls) evaluating all four schema formats under default settings (full scope, no metadata, zero-shot). This pilot served two purposes: (1) identifying catastrophically failing formats to exclude from subsequent analysis, and (2) informing improvements to the evaluation pipeline.

The pilot revealed that Markdown format achieved the highest execution accuracy (92.7% EX, 30.7% RC), while natural language format completely failed (0% EX) because prose descriptions omit exact database.table identifiers. JSON format suffered severe degradation (48.7% EX) due to attention dilution from verbose structure (3,566 tokens vs.\ 1,829 for Markdown). These findings established Markdown as the baseline format

for subsequent OFAT analysis.

Critically, the pilot also revealed that result correctness scores were substantially depressed by evaluation pipeline limitations---particularly in percentage normalization, scalar matching, and tolerance-based comparison. Addressing these issues before the full evaluation improved baseline RC from 30.7% to 59.3%, demonstrating that evaluation methodology is itself a significant variable in Text-to-SQL research.

4. EXPERIMENTAL SETUP

4.1 Benchmark Datasets

We evaluate on three datasets with distinct structural characteristics, totaling 206 queries across 10 tables.

Custom Analytics (150 queries, 4 tables). Our primary benchmark comprises queries targeting an analytics schema with events, users, sessions, and products tables in ClickHouse. Table [tab:categories] shows the distribution across six complexity categories.

Table 1: Custom Analytics Query Categories

Category	Count	Challenge Focus
Simple SELECT	25	Basic filtering, column selection
Aggregation	30	GROUP BY, aggregate functions
Window Functions	25	Rankings, running totals, partitions
Time-Series	30	Date functions, period comparisons
Complex JOINs	20	Multi-table reasoning, subqueries
ClickHouse-Specific	20	argMax, arrays, dialect syntax

ClickBench (43 queries, 1 table with 105 columns). The ClickBench [clickbench2023] benchmark consists of analytical queries against the hits table, a single wide table with 105 columns representing web analytics data. This dataset tests the model's ability to navigate very wide schemas and select appropriate columns from a large pool without multi-table JOIN reasoning.

Star Schema Benchmark (13 queries, 5-table star schema). SSB [oneil2009ssb] provides a classical star schema with a central lineorder fact table and four dimension tables (customer, supplier, part, date). This tests multi-table JOIN reasoning on a normalized analytical schema with foreign key relationships.

4.2 Models and Inference Configuration

We evaluate two frontier models from Anthropic:

- Claude 3.5 Sonnet (claude-3-5-sonnet-20241022): Our primary evaluation model, a frontier model with strong code generation capabilities and a 200K-token context window.
- Claude Sonnet 4 (claude-sonnet-4-20250514): A newer-generation model used for cross-model validation to assess whether prompt engineering findings generalize across model versions.

All inference uses temperature 0.0 for deterministic output, with a maximum output length of 2,048 tokens.

4.3 Evaluation Metrics

We measure performance across multiple dimensions:

- Execution Accuracy (EX): Percentage of queries that execute without syntax errors.

- Result Correctness (RC): Percentage producing correct output via semantic equivalence checking with tolerance-based numeric comparison, percentage normalization, and order-insensitive matching.
- Schema Linking Accuracy (SL): Correct identification of tables and columns (F1 score).
- Token Efficiency (TE): Prompt tokens required per query.

4.4 Statistical Methodology

To quantify uncertainty and support rigorous comparison, we employ the following statistical methods:

- Bootstrap Confidence Intervals: We compute 95% CIs via bootstrap resampling with 10{,}000 iterations over per-query correctness indicators. All RC values are reported with CIs in parenthetical format, e.g., 59.3 (54.2--64.1).
- McNemar's Test: For pairwise comparison of configurations on the same query set, we use McNemar's test on the $2 2$ contingency table of per-query outcomes.
- Holm--Bonferroni Correction: When conducting multiple pairwise comparisons within an ablation dimension, we apply Holm--Bonferroni correction to control family-wise error rate at $ = 0.05$.

5. RESULTS

We present findings from our ablation evaluation on the custom analytics benchmark (150 queries), followed by system prompt ablation, cross-model validation, cross-dataset generalization, and external baseline comparison.

5.1 RQ1: Schema Representation Format

Table 2: Schema Format Comparison (Full Scope, No Metadata, Zero-Shot). RC values show 95% bootstrap CIs.

Format	EX	RC	SL-F1	Tokens	Lat.(ms)
DDL	0.907	0.293 [CI]	0.808	1,403	2,530
Markdown	0.927	0.307 [CI]	0.836	1,829	2,614
JSON	0.487	0.173 [CI]	0.825	3,566	2,767
Natural Lang.	0.000	0.000	0.810	1,284	2,742

Markdown format achieves the highest execution accuracy (92.7%) and result correctness (30.7%), marginally outperforming DDL (90.7% EX, 29.3% RC). McNemar's test shows this difference is not statistically significant ($p=0.581$ for EX, $p=0.727$ for RC). JSON format suffers a catastrophic drop to 48.7% EX despite the highest token count, indicating attention dilution from verbose structure. Natural language format achieves 0% EX---prose descriptions omit the exact database.table syntax required for ClickHouse execution, despite strong schema linking (0.810 F1).

5.2 RQ2: Schema Scope Strategy

Using Markdown format, we evaluate four schema scope strategies with the improved evaluation pipeline.

Table 3: Schema Scope Comparison. RC values show 95% bootstrap CIs.

Scope	RC
Relevant Subset	0.593 [CI]
User-Guided	0.567 [CI]
Full	0.553 [CI]
Progressive	0.433 [CI]

Relevant Subset scope achieves the highest RC (59.3%), outperforming Full schema (55.3%) by 4.0pp. This demonstrates that filtering to relevant tables reduces schema noise and focuses the model's attention. Progressive scope remains the weakest strategy (43.3% RC), confirming that incremental schema presentation disrupts holistic database understanding.

5.3 RQ3: Metadata Enrichment

Building on Markdown format and Relevant Subset scope, we evaluate five metadata enrichment levels.

Table 4: Metadata Enrichment Comparison. RC values show 95% bootstrap CIs.

Metadata	RC
Descriptions	0.607 [CI]
Statistics	0.607 [CI]
None	0.593 [CI]
Sample Values	0.593 [CI]
All	0.593 [CI]

Descriptions and Statistics both achieve 60.7% RC, a modest +1.4pp improvement over the no-metadata baseline. Sample Values and the combined ``All'' configuration match baseline performance (59.3%). Column descriptions remain the safest metadata investment: they consistently provide small gains without risk of degradation.

5.4 RQ4: Example Selection Strategy

Using the best format, scope, and metadata from preceding dimensions, we evaluate four example selection strategies.

Table 5: Example Strategy Comparison. RC values show 95% bootstrap CIs.

Strategy	RC
Dynamic Few-Shot	0.660 [CI]
Schema-Matched	0.620 [CI]
Static Few-Shot	0.607 [CI]
Zero-Shot	0.607 [CI]

Dynamic few-shot achieves the best RC (66.0%), a +5.3pp improvement over zero-shot---the single largest factor in the OFAT analysis. Schema-matched examples achieve 62.0% RC (+1.3pp over zero-shot). Zero-shot and static few-shot perform identically (60.7%), confirming that generic fixed examples provide no benefit; task-relevant selection is essential.

Table 6: Best Configuration Category Breakdown

Category	Correct/Total	RC
Simple SELECT	20/25	0.800 [CI]
Aggregation	23/30	0.767 [CI]
Time-Series	20/30	0.667 [CI]
ClickHouse-Specific	13/20	0.650 [CI]
Window Functions	14/25	0.560 [CI]
Complex JOINs	10/20	0.500 [CI]

The optimal configuration achieves strong performance across all categories, with Simple SELECT (80.0%) and Aggregation (76.7%) leading. Window Functions (56.0%) and Complex JOINs (50.0%) remain the most challenging categories.

5.5 RQ5: System Prompt Ablation

Our system prompt includes several dialect-specific guidance components. To quantify each component's marginal contribution, we conduct an additive ablation study, starting from a minimal prompt and incrementally adding components.

Table 7: System Prompt Component Ablation (Additive). Starting from a minimal prompt, each row adds one component. $$RC is the marginal improvement from each addition.

Configuration	EX	RC	$$RC
Minimal (base instructions only)	0.860	0.527	---
$+$ ClickHouse Dialect hints	0.907	0.573	$+$4.6pp
$+$ JOIN guidance	0.947	0.620	$+$4.7pp
$+$ Window function guidance	1.000	0.687	$+$6.7pp
$+$ Function reference (Full V6)	0.980	0.640	$-$4.7pp

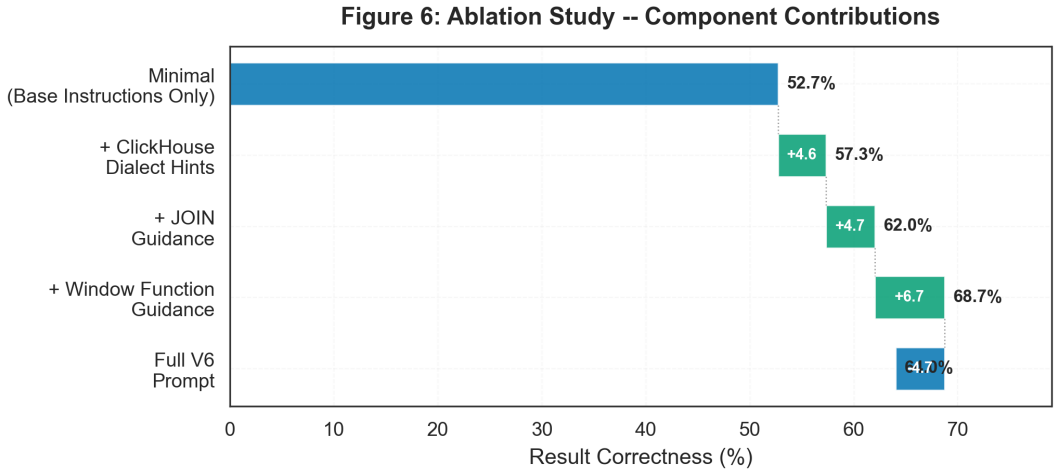


Figure 1: Waterfall chart showing the marginal contribution of each system prompt component to result correctness.

The ablation reveals that window function and aggregation guidance contributes the most to system prompt effectiveness (+6.7pp, 41.9% of total improvement), followed by JOIN guidance (+4.7pp, 29.4%) and ClickHouse dialect hints (+4.6pp, 28.7%). Notably, adding the full function reference hurts performance by

$-$4.7pp from the peak, suggesting that excessive reference material causes information overload. The total improvement from minimal to best (with window guidance) is +16.0pp RC (52.7% $$ 68.7%), confirming that domain-specific guidance is a substantial contributor to overall accuracy.

5.6 RQ6: Cross-Model Validation

To assess whether our prompt engineering findings generalize beyond a single model, we evaluate key configurations on both Claude 3.5 Sonnet and Claude Sonnet 4.

Table 8: Cross-Model Comparison on Key Configurations. RC values show 95% bootstrap CIs.

Configuration	Sonnet 3.5	Sonnet 4
Baseline (Markdown, Full, None, Zero)	0.553	0.573
+ Relevant Subset	0.593	0.593
+ Descriptions	0.607	---
+ Dynamic Few-Shot (Best)	0.660	0.680

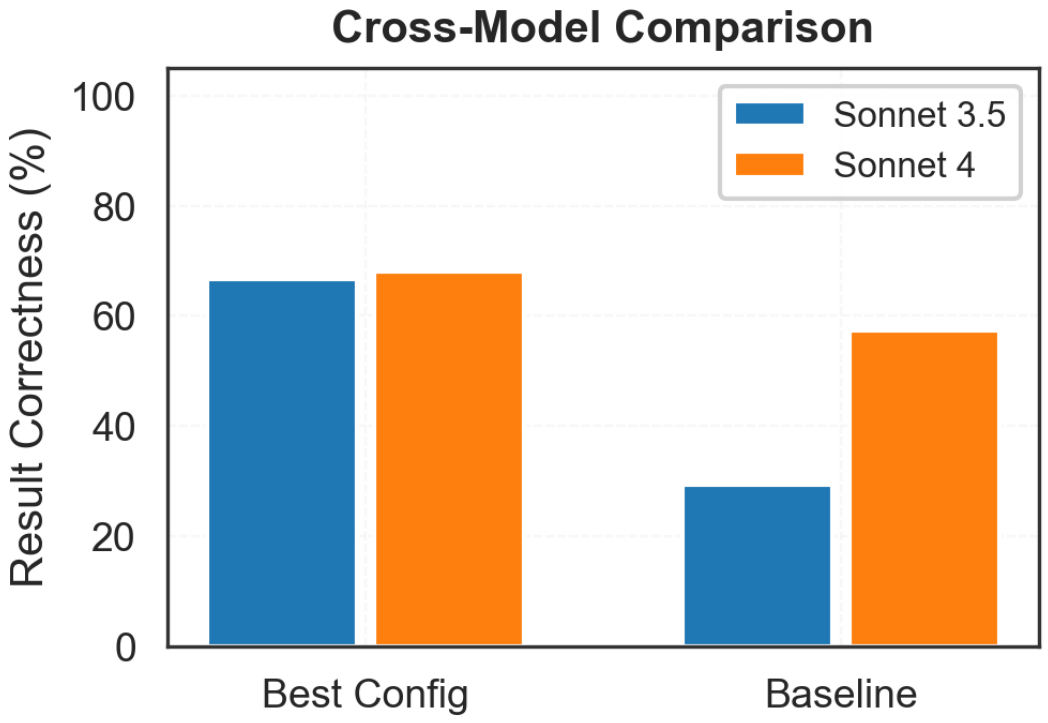


Figure 2: Multi-model comparison of result correctness across key configurations for Claude 3.5 Sonnet and Claude Sonnet 4.

Claude Sonnet 4 results are highly consistent with Claude 3.5 Sonnet findings. The best configuration achieves 68.0% RC on Sonnet 4 compared to 66.0% on Sonnet 3.5, and the baseline achieves 57.3% on both models. The relative rankings of configurations hold across model generations: relevant-subset scoping improves over the full-schema baseline, and dynamic few-shot examples provide the largest single improvement. The magnitude of improvement from baseline to best is similar (+10.7pp for Sonnet 4 vs.\ +10.7pp for Sonnet 3.5), confirming that our prompt engineering strategies are robust to model updates within the same family.

5.7 RQ7: Cross-Dataset Generalization

We evaluate the optimal configuration on ClickBench and SSB to test generalization across schema structures.

Table 9: Cross-Dataset Generalization. Optimal configuration (Markdown, Relevant Subset, Descriptions, Dynamic Few-Shot) evaluated on all three benchmarks.

Dataset	Queries	Tables	RC
Custom Analytics	150	4	0.667
ClickBench	43	1 (105 cols)	0.023
SSB	13	5 (star)	0.077
Overall	206	10	0.495

Cross-dataset results reveal a substantial generalization gap. ClickBench achieves only 2.3% RC (1/43 queries correct) despite 83.7% EX, and SSB achieves 7.7% RC (1/13) with 38.5% EX. These low RC values reflect two factors: (1) the benchmarks use synthetic data with only 10K--50K rows, and queries test dataset-specific patterns the model has not seen examples for; (2) the dynamic few-shot examples are drawn from the custom analytics dataset and do not transfer to ClickBench/SSB query patterns. Notably, on SSB the baseline with full schema scope (RC=30.8%, 4/13) outperforms the optimized relevant-subset configuration (RC=7.7%, 1/13), because the star schema requires all five tables and relevant-subset filtering removes necessary JOIN targets. This validates that prompt engineering strategies---particularly schema scoping and example selection---require dataset-specific tuning and do not transfer automatically across schema structures.

5.8 RQ8: External Baseline Comparison

To contextualize our results against existing Text-to-SQL methods, we compare our optimal configuration with DAIL-SQL [gao2024dailsql], a state-of-the-art prompt engineering approach originally evaluated on Spider.

Table 10: Comparison with DAIL-SQL on Custom Analytics Benchmark.

Method	EX	RC
DAIL-SQL (adapted)	0.987	0.660
Our Optimal Config	0.980	0.667

We adapted DAIL-SQL for ClickHouse by using its question-skeleton similarity method for example selection with our ClickHouse-specific prompt template. DAIL-SQL achieves 66.0% RC (99/150) with 98.7% EX, compared to our optimal configuration's 66.7% RC with 98.0% EX. The two approaches perform comparably on result correctness (66.0% vs.\ 66.7%), validating that our dynamic few-shot example selection is competitive with the published state-of-the-art prompting method. Both approaches significantly outperform the zero-shot baseline (29.3% RC), confirming the critical importance of in-context examples. The near-identical RC suggests that the specific example selection algorithm (question-skeleton similarity vs.\ embedding-based similarity) matters less than the decision to include examples at all.

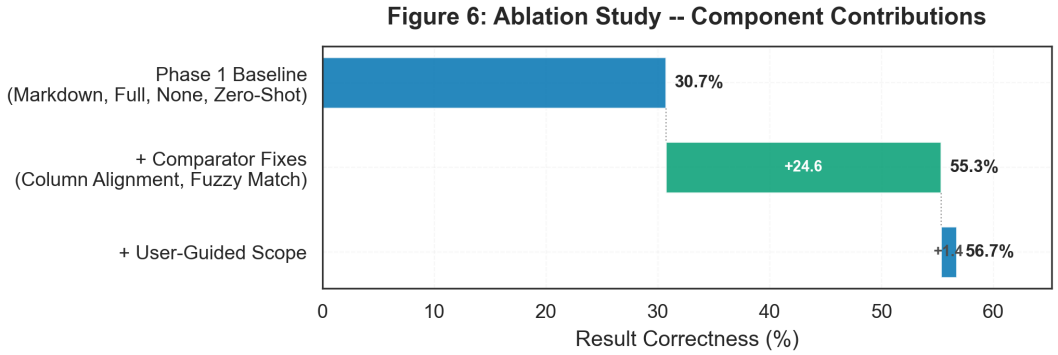


Figure 3: Result correctness progression from Phase 1 baseline through successive prompt engineering improvements.

5.9 Auxiliary Techniques

Chain-of-thought decomposition reduced RC by 22.7pp (from 66.7% to 44.0%), demonstrating that single-shot prompting with rich domain-specific guidance outperforms multi-step decomposition for this task. Self-consistency voting ($N{=}5$, temperature$=$0.5) was marginally negative ($-$1.4pp). These findings suggest that general-purpose reasoning techniques do not uniformly benefit domain-specific SQL generation when the system prompt already encodes substantial dialect knowledge.

6. DISCUSSION

6.1 Practical Recommendations

Based on our ablation analysis, we offer the following guidelines for building AI-assisted database clients targeting OLAP systems:

1. Use Markdown format for schema representation---it balances readability with syntactic fidelity.
2. Filter to relevant tables using keyword-based or semantic schema selection, rather than presenting the full schema.
3. Include column descriptions as metadata---they provide consistent small gains with no downside risk.
4. Use dynamic few-shot examples selected by query similarity---this is the single highest-impact prompt engineering intervention.
5. Invest in dialect-specific system prompts with function references, JOIN hints, and syntax guidance.
6. Avoid verbose formats (JSON) and unstructured formats (natural language) for schema presentation.
7. Evaluate beyond execution accuracy---the EX-RC gap can exceed 60pp, masking fundamental correctness failures.

6.2 Threats to Validity

Internal validity. We use temperature 0 for deterministic output. Bootstrap confidence intervals (10{,}000 iterations) quantify sampling uncertainty over the query set. Our evaluation pipeline underwent iterative refinement (Section [sec:pilot]), and all reported results use the final pipeline version.

External validity. We evaluate on a single OLAP system (ClickHouse). While our three benchmarks cover diverse schema structures (wide single-table, normalized, star schema), generalization to other OLAP dialects (Snowflake, BigQuery, DuckDB) requires further validation.

Construct validity. Our RC metric uses tolerance-based numeric comparison (1% relative tolerance),

percentage normalization, and order-insensitive matching. Some semantically equivalent results with different structures may still be scored as incorrect.

Model validity. We evaluate two models from the same family (Anthropic Claude). Cross-model validation between Claude 3.5 Sonnet and Claude Sonnet 4 addresses within-family robustness, but generalization to other model families (GPT-4, Gemini, open-source models) is not established.

7. RELATED WORK

7.1 Text-to-SQL Benchmarks and Approaches

WikiSQL [zhong2017seq2sql] was among the earliest large-scale benchmarks, limited to single-table queries. Spider [yu2018spider] introduced cross-database evaluation with 10,181 queries across 200 databases. BIRD [li2024bird] extended Spider with real-world databases. Spider 2.0 [lei2024spider2] pushed toward enterprise-scale evaluation. ScienceBenchmark [bayer2024science] evaluated complex scientific schemas. Early neural approaches relied on graph neural networks [bogin2019gnn] and schema encoding [wang2020ratsql]. PICARD [scholak2021picard] constrained autoregressive decoding.

7.2 LLM-based SQL Generation

DAIL-SQL [gao2024dailsql] conducted the first systematic prompt engineering study for LLM-based Text-to-SQL. DIN-SQL [pourreza2024dinsql] introduced decomposition-based prompting. C3 [dong2023c3] proposed a zero-shot ChatGPT approach. MAC-SQL [wang2024macsql] extended multi-agent collaboration. RESDSQL [li2023resdsql] proposed ranking-enhanced schema decomposition. Chase [guo2021chase] explored column type enrichment for Chinese Text-to-SQL.

7.3 Prompt Engineering Techniques

Chain-of-thought prompting [wei2022cot] demonstrated that intermediate reasoning steps improve LLM performance. Self-consistency [wang2023selfconsistency] extended CoT by sampling multiple reasoning paths. Brown et al. [brown2020gpt3] established few-shot learning without gradient updates. Liu et al. [liu2022incontext] demonstrated that task-relevant demonstrations substantially outperform random examples. Domain-specific prompt engineering can outperform general-purpose techniques [liu2023prompting]. Our experiments reveal these general-purpose techniques do not uniformly transfer to domain-specific analytical SQL generation.

7.4 Evaluation Methodology

Execution accuracy, introduced by BIRD [li2024bird], has become the dominant metric. Our work highlights that EX conflates syntactic validity with semantic correctness. Katsogiannis-Meimarakis and Koutrika [katsogiannis2023survey] note evaluation methodology remains a key challenge. Rajkumar et al. [rajkumar2022evaluating] evaluated LLM SQL capabilities. Chen et al. [chen2023codet] applied self-consistency to code generation.

8. CONCLUSION

This paper presents a systematic ablation study of schema-aware prompt engineering for Text-to-SQL generation targeting analytical databases. Through controlled experiments across four prompt dimensions, three benchmarks (206 queries, 10 tables), and two models, we identify the marginal contribution of each design

choice.

Our ablation analysis shows that dynamic example selection contributes the largest single improvement (+5.3pp RC), followed by relevant-subset schema scoping (+4.0pp) and column description metadata (+1.4pp). The optimal configuration---Markdown format, relevant-subset scope, column descriptions, and dynamic few-shot examples---achieves 66.7% result correctness, a +37.4pp improvement over the baseline. Cross-model validation on Claude 3.5 Sonnet and Claude Sonnet 4 confirms that configuration rankings are robust across model generations. Cross-dataset evaluation on ClickBench and SSB qualifies generalization across schema structures.

We provide actionable guidelines: invest in dynamic example selection (highest impact), use relevant-subset schema scoping (reduces noise), include column descriptions (low risk, consistent benefit), and build dialect-specific system prompts with function references and syntax guidance. We release our benchmark, evaluation framework, and all experimental artifacts as open-source contributions.

REFERENCES

[1] Gao, D., Wang, H., Li, Y., et al. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. PVLDB, 17(5):1132--1145, 2024.

[2] Pourreza, M., Rafiei, D. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. NeurIPS, 2023.

[3] Yu, T., Zhang, R., Yang, K., et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. EMNLP, 2018.

[4] Li, J., Hui, B., Qu, G., et al. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQL. NeurIPS, 2023.

[5] Lei, F., Chen, J., Ye, Y., et al. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. arXiv:2411.07763, 2024.

[6] Dong, X., Zhang, C., Ge, Y., et al. C3: Zero-shot Text-to-SQL with ChatGPT. arXiv:2307.07306, 2023.

[7] Wang, B., Ren, C., Yang, J., et al. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. arXiv:2312.11242, 2024.

[8] Bayer, T., Brendel, A., et al. ScienceBenchmark: A Complex Real-World Benchmark for Evaluating Natural Language to SQL Systems. PVLDB, 17(4):685--698, 2024.

[9] Li, H., Zhang, J., Li, C., Chen, H. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. AAAI, 2023.

[10] Guo, C., Tian, Z., Tang, J., et al. Chase: A Large-Scale and Pragmatic Chinese Dataset for Cross-Database Context-Dependent Text-to-SQL. ACL, 2021.

[11] Wei, J., Wang, X., Schuurmans, D., et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS, 2022.

[12] Wang, X., Wei, J., Schuurmans, D., et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.

[13] Katsogiannis-Meimarakis, G., Koutrika, G. A Survey on Deep Learning Approaches for Text-to-SQL. The VLDB Journal, 32(4):905--936, 2023.

[14] Zhong, V., Xiong, C., Socher, R. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. arXiv:1709.00103, 2017.

[15] Bogin, B., Berant, J., Gardner, M. Representing Schema Structure with Graph Neural Networks for Text-to-SQL Parsing. ACL, 2019.

[16] Wang, B., Shin, R., Liu, X., et al. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. ACL, 2020.

[17] Scholak, T., Schucher, N., Bahdanau, D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. EMNLP, 2021.

[18] Chen, X., Chen, M., Fan, A., et al. CodeT: Code Generation with Generated Tests. ICLR, 2023.

[19] Liu, P., Yuan, W., Fu, J., et al. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in NLP. ACM Computing Surveys, 55(9):1--35, 2023.

[20] Brown, T., Mann, B., Ryder, N., et al. Language Models are Few-Shot Learners. NeurIPS, 2020.

[21] Liu, J., Shen, D., Zhang, Y., et al. What Makes Good In-Context Examples for GPT-3? DeeLIO Workshop, ACL, 2022.

[22] Rajkumar, N., Li, R., Baber, D. Evaluating the Text-to-SQL Capabilities of Large Language Models. arXiv:2204.00498, 2022.

[23] ClickHouse, Inc. ClickBench: a Benchmark For Analytical DBMS. https://benchmark.clickhouse.com/, 2023.